

1. Origin and History	1
2. Community Contributions and Versions	1
3. What John Does: Primary Functions	2
4. Techniques Used by John	2
5. File Storage and Structure	3
6. Supported Hash Formats	3
7. Use Cases: Real-World Scenarios	3
8. How to Use John the Ripper	3
9. Extensibility and Customization	4
10. Advanced Features	4
11. Future Outlook	4
Cheat Sheet.....	5

1. Origin and History

John the Ripper started its journey in the early 1990s. It was created by *Alexander Peslyak*, better known as *Solar Designer*. He is the founder of the Openwall Project, a cybersecurity initiative known for developing various security tools and resources. The primary purpose of John was to help identify weak passwords in Unix-based systems, but it rapidly evolved due to the community's input and growing security needs.

John was initially just a simple password cracker for Unix systems. However, with time, various developers began adding support for more hash types and platforms, making it a universal password auditing tool. The community's contributions played a crucial role in expanding its capabilities. Today, John the Ripper is available in two main editions:

1. **Community Edition** (open-source, free).
2. **John the Ripper Pro** (commercial version with more features).

2. Community Contributions and Versions

There are several branches and specialized builds of John, the most notable being the *Jumbo Patch*. The Jumbo version includes more hash types, additional functionalities, and better optimizations. Various developers, from universities, research labs, and independent contributors, have added support for modern encryption standards, GPU optimizations, and parallel processing techniques.

For example:

- **JimF** is a major contributor who has worked on DES formats and contributed heavily to the Jumbo Patch.
- **magnumripper** is another key contributor known for integrating advanced formats and performance improvements.

These contributions have made John a favorite among security researchers, system administrators, and pentesters.

3. What John Does: Primary Functions

John the Ripper's main function is to crack password hashes. It does so by:

1. **Identifying the hash type:** John can automatically recognize various hash formats, from traditional Unix hashes (DES, MD5) to complex hashes like NTLM or Kerberos.
2. **Cracking the hashes:** John uses multiple strategies (dictionary attacks, brute force, rule-based attacks) to convert the hash back into plaintext.

It's effective for auditing password strength and finding weak passwords within large datasets.

4. Techniques Used by John

Here's a more detailed look at the methods John uses:

- **Dictionary Attacks:**
 - Uses a wordlist file containing potential passwords (like `password.lst`).
 - John checks each word from the list against the password hash.
 - It can also apply common transformations (like `password123!`) using *predefined rules*.
- **Brute Force Attacks:**
 - Generates and tests all possible combinations of characters.
 - Starts with short combinations and moves to longer ones.
 - Effective for short or low-complexity passwords but resource-intensive.
- **Rule-Based Attacks:**
 - John has a powerful rule engine that applies mutations to words.
 - It can add numbers, swap cases, or replace letters (password to `p@ssw0rd`).
 - Rules can be custom defined in `john.conf`.
- **Incremental Mode:**
 - This is the most complex mode.
 - Uses statistical analysis to guess likely combinations.
 - Can test billions of possibilities quickly but requires more system resources.

5. File Storage and Structure

When you install or compile John, its directory is organized like this:

- **/run**: Holds the main John executable (`john`) and other configuration files (`john.conf`).
- **/doc**: Documentation and help files explaining John's features.
- **/src**: Contains the C source code files.
- **/test**: Test cases and sample hash files used for validating the build.
- **/wordlists**: Wordlists (e.g., `password.lst`).

Key configuration files:

- **john.conf**: This configuration file controls the rules, modes, and other settings.
- **john.pot**: Stores successfully cracked passwords.
- **john.rec**: Stores the session state to resume an interrupted attack.
- **john.log**: Logs the details of what John is doing.

6. Supported Hash Formats

John supports a vast array of hash formats. Some popular ones include:

1. **Traditional Unix DES and MD5.**
2. **SHA-1, SHA-256, SHA-512.**
3. **LM and NTLM (Windows hashes).**
4. **Kerberos 5.**
5. **Blowfish.**
6. **BCrypt.**
7. **Cisco PIX, MD5, and SHA.**

There are more specialized formats like *MySQL*, *Oracle*, and *PDF hashes* supported by the Jumbo edition.

7. Use Cases: Real-World Scenarios

- **Penetration Testing**: John is extensively used by pentesters to find weak passwords during security audits. It helps test the strength of passwords and highlight potential vulnerabilities.
- **System Administration**: Sysadmins use it to perform internal audits, ensuring that none of their users have weak or easily guessable passwords.
- **Digital Forensics**: During investigations, forensic analysts may use John to recover passwords from encrypted files or devices, aiding in evidence collection.
- **Educational and Research Purposes**: John is often used in academic settings to teach students about cryptography and password security.

8. How to Use John the Ripper

Let's take some example commands and their descriptions:

- **Starting with a wordlist:**

```
john --wordlist=password.lst hashes.txt
```

- Uses the wordlist `password.lst` to crack the hashes in `hashes.txt`.

- **Resuming an interrupted session:**

`john --restore`

- Restores the last interrupted session using the `john.rec` file.

- **Using brute force:**

`john --incremental hashes.txt`

- Runs an incremental brute force attack on `hashes.txt`.

9. Extensibility and Customization

John's power comes from its configurability. You can customize:

- **Rules in `john.conf`:** Modify how wordlists are used and transformed.
- **Custom Wordlists:** You can use any wordlist or create your own.
- **Modular Design:** Developers can write plugins or add new hash formats.

10. Advanced Features

John can use GPUs (using OpenCL) for faster cracking, making it ideal for modern GPUs in combination with tools like *hashcat*. It also supports *distributed cracking*, meaning you can spread the load across multiple machines.

11. Future Outlook

John is expected to continue growing, with improvements focused on speed and expanding support for new encryption standards. The community's involvement ensures that it stays relevant and powerful in the cybersecurity landscape.

JtR CHEAT SHEET

This cheat sheet presents tips and tricks for using JtR

JtR Community Edition - Linux

Download the JtR Bleeding Jumbo edition with improved capabilities and other goodies.

```
git clone
https://github.com/magnumripper/JohnTheRipper -b bleeding-jumbo
```

```
Compile JtR and enable/disable required features
cd JohnTheRipper/
cd src/
./configure
make clean && make -s
```

Enable bash completion. add the following line to your ~/.bashrc

```
. <JtR path>/run/john.bash_completion
```

Cracking Modes

Wordlist Mode (dictionary attack)

```
./john --wordlist=password.lst hashfile
```

Mangling Rules Mode (hybrid)

```
./john --wordlist=password.lst -
rules:<rulename> hashfile
```

Incremental mode (Brute Force)

```
./john --incremental hashfile
```

External mode (use a program to generate guesses)

```
./john --external: <rulename> hashfile
```

Loopback mode (use POT as wordlist)

```
./john --loopback hashfile
```

Mask mode (read MASK under /doc)

```
./john --mask=?1?1?1?1?1?1?1?1 -1=[A-Z]
hashfile -min-len=8
```

Hybrid Mask mode

```
./john -w=password.lst -
mask='?1?1?w?1?1' hashfile
```

Markov mode (Read MARKOV under /doc).
First generate Markov stats:

```
./calc_stat wordlist markovstats
```

Then run:

```
./john -markov:200 -max-len:12 hashfile
--mkv-stats=markovstats
```

Prince mode (Read PRINCE under /doc)

```
./john --prince=wordlist hashfile
```

Most modes have Maxlen=13 in John.conf but it can be overwritten with -max-len=N up to 24

Multiple CPU or GPU

List OpenCL devices and get the device id

```
./john --list=opencl-devices
```

List formats supported by OpenCL

```
./john --list=formats --
format=opencl
```

Multiple GPU's

```
./john hashes --
format:<openclformat> --wordlist:<>
--rules:<> --dev=0,1 --fork=2
```

Multiple CPU's (e.g., 4 cores)

```
./john hashes --wordlist:<> --
rules:<> --dev=2 --fork=4
```

Rules

--rules:Single

--rules:Wordlist

--rules:Extra

--rules:Jumbo (all the above)

--rules:KoreLogic

--rules:All (all the above)

Incremental Modes (Brute Force)

--incremental:Lower (26 char)

--incremental:Alpha (52 char)

--incremental:Digits (10 char)

--incremental:Alnum (62 char)

Incremental mode with new charsets

Create a new charset based on john.pot

```
./john --make-charset=charset.chr
```

Create a new entry in John.conf to accommodate the new charset

```
# Incremental modes
[Incremental:charset]
File = $JOHN/charset.chr
MinLen = 0
MaxLen = 31
CharCount = 95
```

Run JtR with the new charset

```
./john --incremental=charset hashfile
```

JtR Community Wiki

<http://openwall.info/wiki/john>

Documentation under doc folder

Matt Weir Blog

<http://reusablesec.blogspot.ch/>

Simple Rule in John.conf

```
[List.Rules:Tryout]
```

```
l  
u  
c  
l r  
l Az"2015"  
d  
l A0"2015"  
A0"#Az"#
```

Details

convert to lowercase

```
l
```

convert to uppercase

```
u
```

#capitalize

```
c
```

#lowercase the word and reverse it (palindrome)

```
l r
```

#lowercase the word and append at end of the word
(Az) the number 2015

```
l Az"2015"
```

duplicate

```
d
```

lowercase the word and prepend at beginning of
the word (A0) the number 2015

```
l A0"2015"
```

Add # to the beginning and end of the word

```
A0"#Az"#
```

Use the Wordlist Rule

Display the password candidates generated with the
mangling rule

```
./john --wordlist=password.lst --stdout  
--rules:Tryout
```

Generate password candidates max length of 8

```
./john --wordlist=password.lst --  
stdout=8 --rules:Tryout
```

Wordlists

Sort a wordlist to use with wordlist rule mode

```
$tr A-Z a-z < SOURCE | sort -u > TARGET
```

Use a POT file to generate a new wordlist

```
cut -d: -f2 john.pot | sort -u > pot.dic
```

Generate candidate passwords for slow hashes.

```
./john --wordlist=password.lst --stdout  
--rules:Jumbo | ./unique -mem=25  
wordlist.uniq
```

Use external mode for complex rules

<http://www.lanmaster53.com/2011/02/creating-complex-password-lists-with-john-the-ripper/>

Generate a wordlist that meets the complexity
specified in the complex filter

```
./john --wordlist=[path to word list] --stdout --  
external:[filter name] > [path to output list]
```

Try sequences of adjacent keys on a keyboard as
candidate passwords

```
john --external:Keyboard hashfile
```

Configuration Items on John.conf

When using both CPU and GPU set this flag

```
Idle = N
```

Hidden Options

```
./john --list=hidden-options
```

Display guesses

```
./john --incremental:Alpha -stdout -  
session=s1
```

Generate guesses with external program

```
crunch 1 6 abcdefg | ./john hashes -  
stdin -session=s1
```

Session and Restore

```
./john hashes -session=name
```

```
./john --restore=name
```

Show cracked passwords

```
./john hashes --pot=<> --show
```

Resources

John-Users Mailing List

<http://www.openwall.com/lists/john-users/>

```
./john hashes --wordlist=password.lst --  
rules:Tryout
```

Simple Wordlist Rules

#lowercase the first character, and uppercase the rest
C

#toggle case of all characters in the word
t

#toggle case of the character in position N
TN

#reverse: "Fred" -> "derF"
r

#duplicate: "Fred" -> "FredFred"
d

#reflect: "Fred" -> "FredderF"
f

#rotate the word left: "jsmith" -> "smithj"
{

#rotate the word right: "smithj" -> "jsmith"
}

#append character X to the word
\$X

#prefix the word with character X
^X

Insert and Delete Wordlist Rules

#Remove the first char from the word
[

#Remove the last char from the word
]

#delete the character in position N
DN

#extract substring from position N for up to M characters
xNM

#insert character X in position N and shift the rest right
iNX

#overstrike character in position N with character X
oNX

Charset and Conversion Wordlist Rules

#shift case: "Crack96" -> "cRaCK(^"
S

#lowercase vowels, uppercase consonants: "Crack96"
-> "CRaCK96"
V

#shift each character right, by keyboard: "Crack96" ->
"Vtsvl07"
R

#shift each character left, by keyboard: "Crack96" ->
"Xeaxj85"
L

Length control

#reject the word unless it is less than N characters
long
<N

#reject the word unless it is greater than N characters
long
>N

#truncate the word at length N
'N

Dictionaries

Generate wordlists from Wikipedia pages: wget
<https://raw.githubusercontent.com/zombie-sam/wikigen/master/wwg.py>

python wwg.py -u
http://pt.wikipedia.org/wiki/Fernando_Pessoa -t 5 -o fernandopessoa -m3

Generate wordlists from Aspell Dict's

aspell dump dicts

sudo apt-get install aspell-es

aspell -d es dump master | aspell -l es
expand | awk 1 RS=" " |\n" > Spanish.dic

Resources

Full Rules Documentation
<http://www.openwall.com/john/doc/RULES.shtml>

Password Analysis and Cracking Kit
<https://thesprawl.org/projects/pack/>

Mangling Rules Generation by Simon Marechal
<http://www.openwall.com/presentations/Pa>